

CHAPTER 05

STYLE 與 CSS 樣式設計

HORAZON

互動媒體設計

什麼是 CSS?

CASCADING STYLE SHEETS (階層式樣式表)

- 負責網頁的**外觀**、**排版**與**視覺效果**。
- HTML 是**骨架** (內容)，CSS 是**樣式** (裝飾)。
- 沒有 CSS 的網頁，就像沒有裝潢的毛胚屋，只有黑白文字。

CSS 的三種寫法

1. **行內樣式 (Inline Style)**：如果不推薦。

```
<h1 style="color: red;">標題</h1>
```

- 缺點：難維護，要把每個標籤都改一遍。

2. **內部樣式 (Internal Style)**：寫在 `<head>` 的 `<style>` 標籤內。

- 缺點：只能給這一頁用。

3. **外部樣式 (External Style)**：寫在獨立的 `.css` 檔案 (推薦！)。

- `<link rel="stylesheet" href="style.css">`
- 優點：可以讓 100 個網頁共用同一個樣式檔。

CSS 語法結構

```
選擇器 {  
    屬性: 設定值;  
    屬性: 設定值;  
}
```

- 選擇器 (Selector)：選到誰？ (例如 `h1` , `p` , `.box`)
- 屬性 (Property)：改什麼？ (例如 `color` , `font-size`)
- 設定值 (Value)：改多少？ (例如 `red` , `16px`)

```
h1 {  
    color: red;  
    font-size: 32px;  
}
```

選擇器 (SELECTORS) - 基礎篇

選擇器	符號	範例	說明
標籤 Element	無	p	選取所有 <code><p></code> 標籤
類別Class	.	.title	選取 <code>class="title"</code> 的元素 (最常用!)
ID	#	#header	選取 <code>id="header"</code> 的元素 (唯一)

選擇器 - 進階篇

除了直接選，還可以選「關係」：

- **後代選擇器 (空白)**：選取裡面的子子孫孫。
 - `.card p` (選取 `.card` 裡面所有的 `p`，不管包幾層)
- **子選擇器 (`>`)**：只選親生兒子 (第一層)。
 - `.menu > li` (只選 `.menu` 直屬的 `li`)
- **屬性選擇器 (`[]`)**：
 - `input[type="text"]` (只選 `type` 是 `text` 的輸入框)
- **群組選擇器 (`,`)**：
 - `h1, h2, h3` (這些標題都套用同樣樣式)

偽類選擇器 (PSEUDO-CLASSES)

描述元素的**狀態**，用冒號 (:) 開頭。

- `:hover`：滑鼠移上去時。
 - `button:hover { background: blue; }`
- `:active`：按下去的那一瞬間。
- `:focus`：輸入框被點擊、準備輸入時。
- `:nth-child(n)`：選第幾個小孩。
 - `li:nth-child(odd)` (選取奇數項，做斑馬紋表格常用)

權重 (SPECIFICITY)

為什麼我的 CSS 沒效？可能是**權重**輸了！

當多個 CSS 規則同時設定同一個元素時，聽誰的？

1. `!important` (最強，但盡量別用)
2. 行內樣式 (`style="..."`)
3. ID (`#id`)
4. Class (`.class`)、偽類 (`:hover`)
5. 標籤 (`div`)
6. 全域 (`*`)

原則：這就是為什麼要多用 Class、少用 ID 的原因 (ID 太強很難覆蓋)。

單位 (UNITS)

- 絕對單位：
 - `px` (像素)：最直觀，但也最死板。
- 相對單位 (推薦)：
 - `em`：相對於「父元素」的字體大小。
 - `rem`：相對於「根元素 (`html`)」的字體大小 (Root em)。
 - `%`：相對於父元素的寬高。
- 視窗單位：
 - `vw` (Viewport Width)：視窗寬度的 1%。
 - `vh` (Viewport Height)：視窗高度的 1%。
 - 常用：`height: 100vh` (滿版畫面)。

文字與排版屬性

- `font-family`：字型 ("微軟正黑體", `sans-serif`) 。
- `line-height`：行高 (如 `1.5` 倍)，閱讀舒適度關鍵。
- `text-align`：對齊 (`center` , `left` , `right` , `justify`) 。
- `text-decoration`：裝飾。
 - `none` (去掉超連結底線)
 - `underline` (底線)
 - `line-through` (刪除線)

盒子模型 (BOX MODEL) - 核心觀念

每個 HTML 元素都是一個**盒子**，由內而外包含：

1. **Content** (內容)：文字或圖片本身。
2. **Padding** (內距)：內容與邊框之間的距離 (留白)。
3. **Border** (邊框)：盒子的框線。
4. **Margin** (外距)：盒子與盒子之間的距離。

BOX SIZING 的陷阱

預設的 `box-sizing: content-box` 很反直覺。
如果你設 `width: 100px`，又加了 `padding: 20px`，
盒子實際寬度會變成 140px！(100 + 20左 + 20右)

解決方案： `BORDER-BOX`

```
* {  
  box-sizing: border-box; /* 內距與邊框都算在 width 裡面 */  
}
```

這樣 `width: 100px` 就是 100px，Padding 會往內擠，不會把盒子撐大。

現代排版神器：FLEXBOX

`display: flex` 解決了所有以往排版 (Float) 的痛苦。

容器 (CONTAINER) 的屬性：

- `flex-direction`：排列方向。
 - `row` (預設，橫排), `column` (直排)。
- `justify-content`：主軸對齊 (橫向)。
 - `center` (置中), `space-between` (分散對齊)。
- `align-items`：交錯軸對齊 (縱向)。
 - `center` (垂直置中), `base-line` (對齊基線)。
- `flex-wrap`：換行。
 - `nowrap` (預設，死都不換行), `wrap` (空間不夠就換行)。

FLEX 項目 (ITEM) 的屬性

- `flex: 1`：大家平分剩餘空間 (自動伸縮)。
- `order`：改變排列順序 (不用改 HTML)。
- `align-self`：只有我不合群，自己對齊別的地方。

```
.container {  
  display: flex;  
  justify-content: center; /* 水平置中 */  
  align-items: center;     /* 垂直置中 */  
  height: 100vh;          /* 滿版高度 */  
}
```

這三行程式碼是網頁置中的聖杯！

現代網格系統：CSS GRID

比 Flex 更強大的二維排版。

Flex 是一條線 (一維)，Grid 是一張表 (二維)。

```
.container {  
  display: grid;  
  /* 產生三個寬度一樣的欄位 */  
  grid-template-columns: 1fr 1fr 1fr;  
  /* 或是指定寬度 */  
  grid-template-columns: 200px auto 100px;  
  
  gap: 20px; /* 格子之間の間距 */  
}
```

背景處理 (BACKGROUNDS)

```
.hero {  
  /* 設定背景圖 */  
  background-image: url('bg.jpg');  
  
  /* 背景不要重複 */  
  background-repeat: no-repeat;  
  
  /* 背景位置置中 */  
  background-position: center;  
  
  /* 背景尺寸：cover (填滿，可能會裁切), contain (完整顯示) */  
  background-size: cover;  
}
```


轉場與動畫 (TRANSITIONS)

不用寫 JS 也能做簡單動畫！

transition：當屬性改變時 (例如 hover)，要花多久時間變過去。

```
button {  
  background: blue;  
  transition: all 0.3s ease; /* 0.3秒，平滑過渡 */  
}  
  
button:hover {  
  background: red; /* 滑鼠移上去，會在 0.3 秒內從藍變紅 */  
  transform: scale(1.1); /* 放大 1.1 倍 */  
}
```

CSS 變數 (VARIABLES)

方便管理主題色，一改全改。

定義變數 (通常在 `:root`) :

```
:root {  
  --main-color: #3498db;  
  --text-color: #333;  
}
```

使用變數：

```
h1 {  
  color: var(--main-color);  
}  
button {  
  background: var(--main-color);  
}
```

響應式網頁設計 (RWD)

Responsive Web Design：讓網頁適應不同裝置。
使用 Media Queries。

```
/* 電腦版樣式 (預設) */  
.box { width: 50%; }  
  
/* 平板 (小於 768px) */  
@media (max-width: 768px) {  
    .box { width: 100%; }  
}  
  
/* 手機 (小於 480px) */  
@media (max-width: 480px) {  
    h1 { font-size: 18px; } /* 字體變小 */  
}
```

實作練習：導覽列 (NAVBAR)

請製作一個響應式導覽列：

1. 使用 `<nav>` 與 `<ul>` 。
2. Flexbox 讓選單橫排 (`display: flex`) 。
3. Hover 效果：滑鼠移上去變色 。
4. Logo 置左，選單 置右 (`justify-content: space-between`) 。

```
nav {  
  display: flex;  
  justify-content: space-between;  
  padding: 1rem;  
  background: #333;  
  color: white;  
}
```

1. CSS 決定網頁的**顏值**。
2. 搞懂 **Box Model** (Border-box) 是基本功。
3. **Flexbox** 是排版主力，一定要熟。
4. **RWD** (Media Query) 是現代網頁必備。
5. 善用 **CSS 變數** 與 **Class** 管理樣式。
6. 用 **Transition** 增加互動質感。

下週，我們將為網頁注入靈魂：JavaScript！